

Common Process Modeling Pitfalls & Fixes

Business process models must be clear, consistent, and well-scoped. Common issues include missing elements (e.g. unnamed start/end), tangled flows, unclear roles, and misused gateways. Below we catalog frequent pitfalls and offer concrete “quick-fix” solutions. Each item includes a brief description with authoritative sources, a checklist of fixes, and a “Bad vs. Good” Mermaid diagram example illustrating the correction.

Scope & Start/End Events

- **Implicit or Missing Start/End:** Every process should have an explicit **Start** and **End** event. Implicit boundaries cause confusion and misinterpretation ¹ ². Without them, it’s unclear how a case is initiated or completed.
- **Fix:** Define at least one Start event (time, message, etc.) and one End event for each process or sub-process ¹ ². If there are alternative triggers, use multiple distinct Start events. Clearly mark success vs. failure end states (separate end events) ³.
- **Checklist:**
 - Include **Start** event symbols at all process entry points ¹ ².
 - Include **End** event symbols where every flow must terminate ¹.
 - Use separate End events for different outcomes (success vs. exception) ³.

Bad Example: (No explicit start/end)

```
flowchart LR
  A[Task A] --> B[Task B] --> C[Task C]
```

Good Example: (With explicit start/end)

```
flowchart LR
  S([Start]) --> A[Task A] --> B[Task B] --> C[Task C] --> E([End])
```

- **Overly Broad or Narrow Scope:** Models sometimes try to capture “everything” (inclusion of too many upstream/downstream tasks) or are too granular (detailing every micro-step) ⁴ ⁵. Either extreme makes the diagram unwieldy.
- **Fix:** Clearly define your goal. Include only those steps needed for the analysis or communication intent ⁴. Start with a high-level “happy path” and refine later as needed. Decompose large processes into subprocesses/call-activities ⁶ ⁷. Use modular or multi-page models (e.g. one page per 5–10 activities) to preserve clarity ⁸.
- **Checklist:**
 - Identify the primary purpose or decision the model supports ⁴.
 - Avoid modeling every detail at once; focus on the core scenario, add detail iteratively ⁴.

- Decompose very large processes into layered subprocesses 6 7 .
- Use subprocesses or call-activities for repeatable fragments 7 .

Diagram Layout & Readability

- **Spaghetti Flows / Crossed Connectors:** Overlapping or long, twisted flows make diagrams hard to read 9 10 . Crossing lines or inconsistent flow direction (“slalom effect”) confuse the reader and mask the main process path 11 12 .
- **Fix:** Arrange elements in a clean, mostly one-directional flow (e.g. left-to-right or top-down) 12 13 . Avoid drawing sequence flows across each other. Use consistent spacing and alignment for shapes 9 . Label only where needed. Highlight the primary scenario first, then add alternate paths 13 .
- **Checklist:**
 - Lay out the diagram so sequence flows run in a consistent direction (e.g. left→right) 12 .
 - Eliminate crossing arrows; reroute paths or break the model into subprocesses if flows cross 9 12 .
 - Ensure similar elements (tasks, events) are sized and spaced uniformly 10 .
 - Clearly distinguish the main (happy) path from alternate scenarios 13 .

Bad Example: (Crossed flows, inconsistent direction)

```
flowchart TD
  A(Start) --> C[Task 1]
  B[Task 2] --> D[Task 3]
  C --> B
  D --> A
```

Good Example: (Clean, one-directional flow)

```
flowchart LR
  S([Start]) --> A[Task 1] --> B{Decision{"Yes?"}} --> C[Task 2] --> E([End])
  A --> D[Task 3] --> E
```

- **Monolithic Diagrams (No Decomposition):** Putting an entire multi-phase process on one page (often with dozens of tasks) destroys readability 14 . Large diagrams hide logic issues (missing steps, loops, inconsistent detail) because users “read to the end” without understanding 15 .
- **Fix:** Break up large processes into layers. Use subprocesses or call activities to encapsulate details 6 7 . Aim for a handful of activities per diagram (7±4 recommended 14). Each subprocess can be expanded in its own diagram.
- **Checklist:**
 - Limit each diagram to a manageable number of elements (around 5–10) 14 .
 - Introduce subprocess nodes to represent complex chunks 6 7 .
 - Use hierarchical models: one high-level map plus detail pages for each subprocess 6 .
 - Ensure consistency in detail: if one part of the process is in-depth, similar sections should be equally detailed 8 .

Naming & Labeling

- **Inconsistent or Vague Labels:** Activities and events should be clearly named. Using nouns for actions or inconsistent verb/noun mix confuses meaning ¹⁶. Gateway labels (e.g. “? Yes/No”) are especially misleading, as gateways represent a decision point, not an action ¹⁷.
- **Fix:** Adopt a consistent naming convention. Use **strong verbs** for tasks (“Review Invoice”), nouns for data or documents ¹⁶. Don’t name gateways or label them with Yes/No (instead, label outgoing flows if needed). Keep names concise (focus on outcomes, details in documentation) ¹⁸.
- **Checklist:**
 - Ensure every task name is action-oriented (verb + object) ¹⁶.
 - Avoid conjunctions or “and/or” in activity names; split into separate tasks if needed ¹⁹.
 - Do not label gateways (they are pure routing); if the gateway’s purpose must be shown, use an outgoing flow label or a preceding decision task ¹⁸.
 - Keep names short but meaningful; move lengthy explanations to tooltips or notes ²⁰.
- **Unlabeled or Misplaced Pools/Lanes:** If roles or departments are not shown in lanes (or mislabeled), readers can’t tell who does what. Conversely, using a pool just to name it after the process is misleading ²¹.
- **Fix:** Use swimlanes (lanes within a pool) or pools to represent distinct actors/participants ²². Place each task or event in the appropriate lane. Do not draw tasks across multiple lanes or leave tasks hanging between lanes ²³. If the diagram is not a collaboration of multiple processes, one pool is enough ²⁴.
- **Checklist:**
 - Identify all participants (roles or departments) and assign each its own swimlane ²².
 - Ensure every task is inside exactly one lane (no task box straddles lane boundaries) ²³.
 - Do not name the pool after the process (the pool represents an actor, not the process itself) ²¹.
 - If there is only one process, use a single pool so flows don’t accidentally cross pools ²⁴.

Bad Example: (No lanes, ambiguous performers)

```
flowchart LR
    A[Enter Data] --> B[Approve Request] --> C[Complete Order]
```

Good Example: (Lanes for roles)

```
flowchart LR
    subgraph Clerk
        A[Enter Data]
    end
    subgraph Manager
        B[Approve Request]
    end
```

```

subgraph Clerk
  C[Complete Order]
end
A --> B --> C

```

Gateway & Decision Logic

- **Missing Gateways:** Tasks should not have multiple outgoing or incoming flows without an explicit gateway. Omitting a decision symbol makes the logic unclear and violates BPMN conventions ²⁵ ²⁶.
- **Fix:** Insert gateways wherever flows split or join. For every task that had multiple outputs, replace it with a preceding gateway (or use a decision task then gateway) ²⁵ ²⁶. Likewise, merge incoming flows with the same type of gateway.
- **Checklist:**
 - Whenever you branch a flow, place an appropriate gateway at the split point ²⁵ ²⁶.
 - Likewise, use a matching gateway to merge flows (e.g. if you split with an exclusive gateway, you should join with an exclusive gateway) ²⁷.
 - Do not draw multiple sequence lines out of a task or to a task without a gateway ²⁵ ²⁶.
 - Label flow conditions (arrow texts) on the outgoing paths to clarify each branch.

Bad Example: (Branching without a gateway)

```

flowchart LR
  Start([Start]) --> Task[Check Stock]
  Task --> Reorder[Initiate Reorder]
  Task --> End([End])

```

Good Example: (Using an exclusive gateway)

```

flowchart LR
  Start([Start]) --> Check[Check Stock]
  Check -->|In Stock| Ship[Process Order]
  Check -->|Out of Stock| Reorder[Initiate Reorder]
  Ship --> End1([End])
  Reorder --> End2[End]

```

- **Wrong Gateway Types / Mismatched Joins:** A split gateway and a join gateway should be of the same type (e.g. do not split with a Parallel Gateway and merge with an Exclusive) ²⁷ ²⁸. Also, using an event-based gateway incorrectly (e.g. with a non-event flow) leads to ambiguity ²⁹.
- **Fix:** Use **exclusive gateway** for mutually exclusive paths (Yes/No decisions), **parallel gateway** for independent parallel tasks, etc. If a flow was split with type X, merge it later with the same type X ²⁷. For event-based gateways, ensure all outgoing flows go to an intermediate event (or include a default timer if needed) ²⁹.

- *Checklist:*
 - Pair every diverging gateway with a matching converging gateway of the same type ²⁸ ²⁷ .
 - Exclusive (XOR) split → Exclusive join; Parallel split → Parallel join ²⁸ ²⁷ .
 - If using event-based gateways, connect each branch to an event; provide a default timer/event path if an expected event never occurs ²⁹ .
 - Avoid naming a gateway; let it be a simple diamond.
- **Too Many or Unnecessary Gateways:** Conversely, overusing gateways (e.g. inserting one at every small decision) can clutter the model ³⁰ . Gateways should only appear where a true branching or merging is needed.
- *Fix:* Only add gateways where the process genuinely branches or synchronizes. Remove trivial gateways that just propagate a single flow. If multiple related gateways follow in sequence, consider consolidating them or modeling the logic inside a business rule task ³¹ .
- *Checklist:*
 - Identify true decision points; use a gateway only when at least two distinct flows diverge or converge.
 - Avoid gateways with only one incoming and one outgoing flow (redundant).
 - Simplify chains of gateways by using a preceding decision task or business rule instead ³¹ .
 - Regularly validate the model to detect “floating” or trivial gateways (many tools can check BPMN compliance).

Events & Exception Handling

- **Missing Exception Paths:** Exceptional flows (errors, cancellations, timeouts) are often omitted. Ignoring error events or boundary events means the model doesn’t reflect real-world failure modes ³² .
- *Fix:* Include error/exception flows explicitly. For example, attach **Error Boundary Events** to tasks where needed, and separate end events for unsuccessful outcomes ³ ³² . Use intermediate events (timers, error, message) to model alternate flows.
- *Checklist:*
 - Identify potential error conditions (system failure, rejection, timeout) in each task.
 - Model these using error end events or intermediate boundary events ³² .
 - Distinguish normal end vs. error end (e.g. separate end events or result codes) ³ .
 - Remove duplicated events (e.g. do not place the same start event inside and outside a subprocess) ³³ .
- **Improper Event Placement:** Placing a start or intermediate event outside of its relevant subprocess or duplicating it in two places leads to redundancy and confusion ³³ . Each event should be scoped correctly.
- *Fix:* Keep start events at the top of the process or subprocess they belong to. If a subprocess has its own triggering event, place it on that subprocess. Do not repeat events both inside and outside the subprocess (they would fire twice) ³³ .

- *Checklist:*

- Place a Start event on exactly one level where the process begins. If a subprocess must start on an event, put that event on the subprocess (not duplicating it outside) ³³ .
- Use intermediate events to catch things happening during a process, and attach them to the correct activity or subprocess.
- Avoid duplicate or dangling events that are not connected to a sequence flow.

Roles & Collaboration

- **Overloaded or Missing Swimlanes:** When a diagram omits lanes or uses them inconsistently, role ambiguity occurs. Conversely, creating many lanes with little content complicates the view.
- *Fix:* Only create swimlanes for distinct performers who actually execute tasks ³⁴ . If a process has only one actor, a single pool with lanes is sufficient ²⁴ . If a lane has no tasks/events, it should be removed or merged. Place tasks that require the same resource in the same lane.

- *Checklist:*

- Use one pool per process ²⁴ .
- Create a lane only if at least one activity/event belongs there ³⁴ .
- Don't split a logical task across two lanes. Each task should live entirely in its performer's lane ³⁵ .
- Do not use swimlanes to represent tools or passive entities (only active roles) ²³ .

- **Pools and Message Flows:** Business context can be lost if collaborations aren't modeled correctly. Within a single process model, do not mix tasks and message flows from outside unless showing explicit collaboration. Also, ensure any external interactions use consistent message flows.

- *Fix:* If modeling a collaboration, use separate pools for each entity and connect them via **Message Flows**. For an internal end-to-end process, model it inside one pool and omit message flows (they shouldn't cross a single pool's boundary) ²⁴ .

- *Checklist:*

- Never draw sequence flow across different pools (use message flow for inter-process communication) ²⁴ .
- If you introduce a caller/receiver (external party), ensure you show all relevant message flows and label them.
- Validate that pools represent different processes; do not put the same process in two pools.

Quality & Validation

- **Ignoring Model Validation:** Skipping syntactic or semantic checks yields BPMN models that may look plausible but contain errors (orphaned tasks, mismatched gateways, etc.) ³⁶ .
- *Fix:* Regularly validate the model with BPMN tools or reviews. Check for connectedness, proper gateway pairing, and that every sequence has a start and end. Perform peer reviews or walkthroughs with stakeholders.

- *Checklist:*

- Use modeling tool validation features (many BPMN tools highlight disconnected elements or rule violations).
- Ensure every task/event/gateway connects to at least one incoming and one outgoing flow (except Start/End) ²⁶ .
- Confirm gateway splits match joins, and that no element is isolated.
- Label diagrams consistently (fonts, colors) and adhere to corporate standards for diagram style.

- **Lack of Testing or Stakeholder Review:** A model built in isolation can miss real-world conditions or stakeholder needs. Without validation against actual data or people, errors persist.

- *Fix:* Treat process models like software: test them. Use simple walk-throughs or simulation (if possible) to ensure the flow behaves as expected. Have end-users or process owners review the model. Incorporate feedback promptly.

- *Checklist:*

- Compare model outcomes to real process data or scenarios. Run through example cases to see if the model produces the right results.
- Engage all key stakeholders in model reviews and incorporate their corrections ³⁷ ³⁸ .
- Revise the model iteratively until it reliably reflects the actual business process.

By addressing these common pitfalls with systematic fixes – clear scope, proper use of BPMN constructs, logical layout, and validation – your process models will be far more robust and useful. Each fix above is supported by BPMN best practices and the literature ¹ ² ²⁵ ¹⁰ .

Sources: Authoritative BPMN modeling guides and practitioner articles ¹ ² ²⁵ ²² have been used to identify pitfalls and recommend fixes. Each snippet and checklist above is grounded in these sources. The Mermaid examples above illustrate “bad” versus “good” practice in simplified form.

¹ ⁷ ⁹ ²¹ ²⁸ ³¹ ³⁶ BPMN Modeling Best Practices

<https://www.trisotech.com/bpmn-modeling-best-practices/>

² ³ ¹² ²³ ²⁴ ²⁶ ²⁷ ³⁴ ³⁵ Low-code Process Automation > Studio Cloud - Authoring environment > Bizagi Studio > Process wizard > Model Process > Best Practices in process modeling

<https://help.bizagi.com/platform/en/best-practices-in-process-mode.htm>

⁴ 5 Common Process Modeling Mistakes and How to Avoid Them | MoreSteam

<https://www.moresteam.com/resources/blogs/process-modeling-mistakes>

⁵ ²² ³⁰ ³² ³⁸ common BPMN modeling mistakes to avoid

https://bpmn.page/article/common_BPMN_modeling_mistakes_to_avoid.html

⁶ ⁸ ¹⁰ ¹¹ ¹³ ¹⁴ ¹⁵ ¹⁶ ¹⁷ ¹⁸ ¹⁹ ²⁰ ²⁵ ²⁹ ³³ Efficient BPMN: from Anti-Patterns to Best Practices > Business Analyst Community & Resources | Modern Analyst

<https://www.modernanalyst.com/Resources/Articles/tabid/115/ID/2438/Efficient-BPMN-from-Anti-Patterns-to-Best-Practices.aspx>

³⁷ Top 6 Business Process Modeling Mistakes

<https://www.modernanalyst.com/Resources/Articles/tabid/115/ID/5819/Top-6-Business-Process-Modeling-Mistakes.aspx>